Statistical Computing HW1

R14621207 王紹宇

2026-09-22

Question 1: Beta(3,2) via Acceptance-Rejection

在這個題目中，由於 target distribution $Beta(3,2)$ 以及 $Uniform(0,1)$ 兩個分配 x 的值域皆為 0 到 1，因此我利用 $Uniform(0,1)$ 作為 candidate distribution，後續將以 $f(x)$ 表示 target distribution，而 $g(x)$ 表示 candidate distribution。

接著我們要計算 $\frac{f(x)}{g(x)}$ 的最大值，從定義中我們得知:

$$\frac{f(x)}{g(x)} = \frac{x^2(1-x)}{B(3,2)} = \frac{x^2 - x^3}{B(3,2)}$$

接著我們對 $\frac{f(x)}{g(x)}$ 微分並設其為 0 即可得到 $\frac{f(x)}{g(x)}$ 的最大值，此最大值發生於當 $x = \frac{2}{3}$ 的時候 $(x = 0)$ 最小值 $(f/g = 0)$。因此我們可以將 $\frac{2}{3}$ 帶入 $\frac{f(x)}{g(x)}$ 後即可得到 acceptance-rejection 公式中的 c

$$\frac{d}{dx}[\frac{f(x)}{g(x)}] = \frac{2x - 3x^2}{B(3,2)} = 0 \Rightarrow x = \frac{2}{3}, 0$$

接著我們便可以利用 acceptance-rejection 生成樣本的概念建立以下的程式碼:

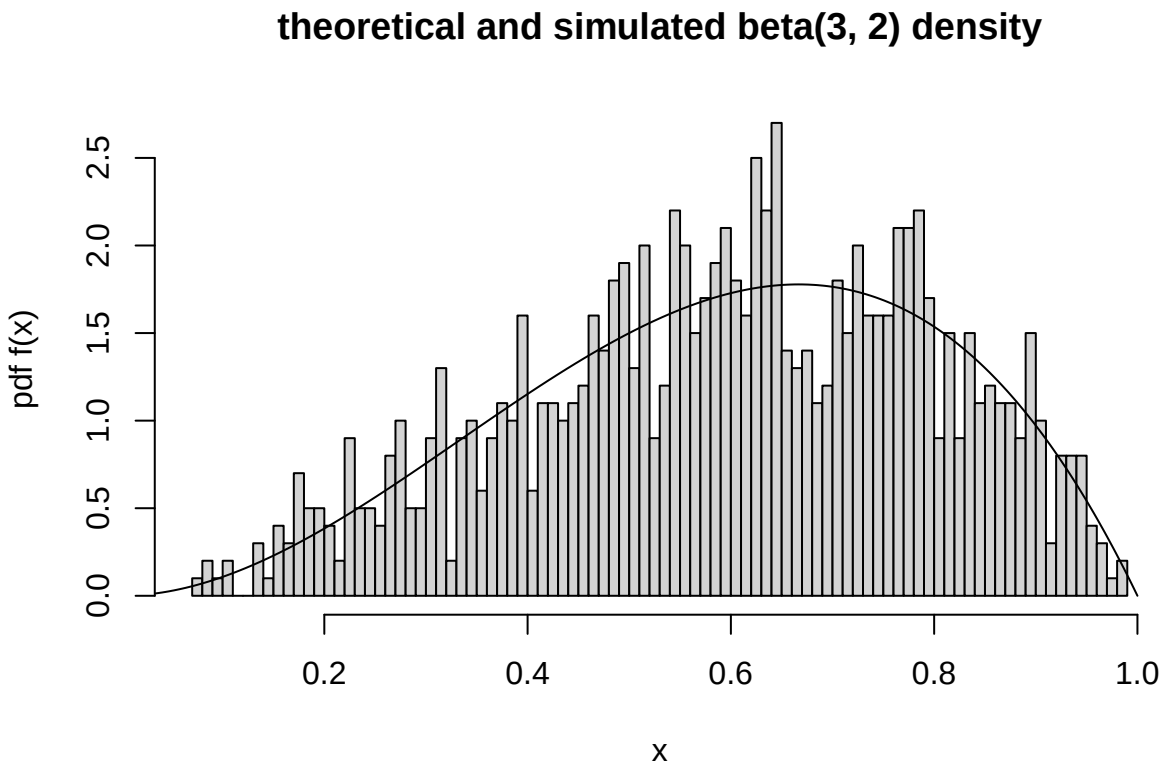
1. 首先我們先建立一個判定是否要接受 candidate distribution 生出來的隨機樣本的判斷式
2. 接著利用迴圈進行 1000 次的隨機變數生成，即可得到 $Beta(3,2)$ 的隨機樣本

```r
# Candidate distribution: unif(0, 1)
beta.sim <- function(alpha, beta, c) {
    stopifnot(alpha >= 1 & beta >= 1)
    while(TRUE) {
        g <- runif(1, 0, 1)
        cu <- runif(1, 0, c)
        f <- g^(alpha - 1) * (1 - g)^(beta - 1) / (beta(alpha, beta))
        if(cu < f) return(g)
    }
}

# find the best c: we got the largest f(x)/g(x) at x = 2/3
# which means the best c we got is
c <- (2/3)^2 * (1/3) / beta(3, 2)
n <- 1000
x <- rep(0, n)
for(i in 1:n) {
    x[i] <- beta.sim(3, 2, c)
}

hist(x, breaks = 100, freq = F, xlab = "x", ylab = "pdf f(x)",
    main = "theoretical and simulated beta(3, 2) density")
```

```r
y <- seq(0, 1, 0.001)
lines(y, dbeta(y, 3, 2))
```



Question 2: Semicircle Distribution via Acceptance-Rejection

這題 target distribution x 的值域為-R 到 R，我認為可以利用 $Uniform(-R, R)$ 作為 candidate distribution 接著我們便計算 $\frac{f(x)}{g(x)}$ 的最大值，計算過程如下

$$f(x) = (2/(\pi \times R^2)) \times \sqrt{R^2 - x^2}$$

$$g(x) = \frac{1}{2R}$$

$$\frac{f(x)}{g(x)} = \frac{4}{\pi \times R} \times \sqrt{R^2 - x^2}$$

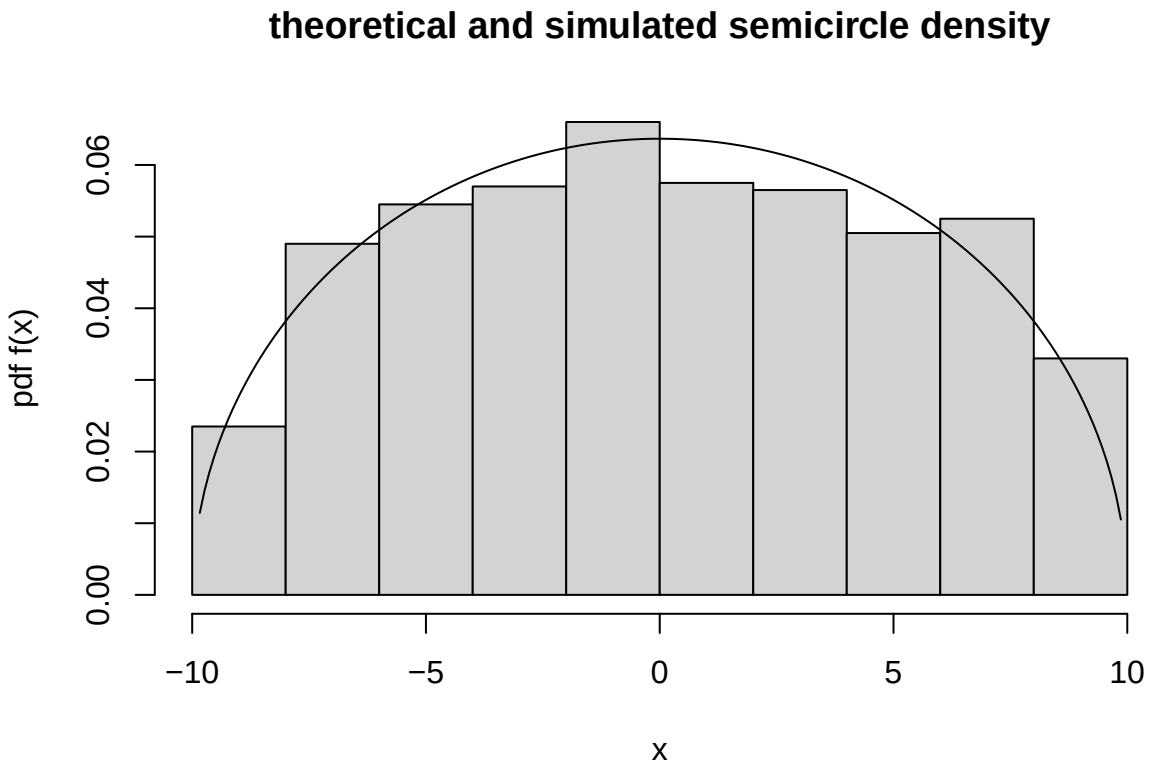
微分過後可以得到當 $x = 0$ 時，$\frac{f(x)}{g(x)}$ 會有最大值 $\frac{4}{\pi}$。接著便將這些數值代入與前一題同樣架構的程式碼中即可得到該分配的隨機樣本。
在這題中，我選擇 $R = 10$。

```r
# f(x) = (2 / (pi*R^2)) * sqrt(R^2 - x^2)
# g(x) = 1 / 2R
# f(x) / g(x) = (4 / (pi * R)) * sqrt(R^2 - x^2)
# f(x) / g(x) has the largest value at 4/pi when x = 0
# c = 4 / pi
semicircle.sim <- function(R) {
    while(TRUE) {
        y <- runif(1, -R, R)
        d <- (4 / (pi * R)) * sqrt(R^2 - y^2)
        u <- runif(1, 0, 4/pi)
```

```
        if(u < d) return(y)
    }
}

dsemicircle <- function(R, x) {
    (2 / (pi*R^2)) * sqrt(R^2 - x^2)
}

n <- 1000
x <- rep(0, n)
R <- 10
for(i in 1:n) {
    x[i] <- semicircle.sim(R)
}
hist(x, freq = F, xlab = "x", ylab = "pdf f(x)",
    main = "theoretical and simulated semicircle density")
y <- seq(min(x), max(x), 0.1)
lines(y, dsemicircle(R = 10, x = y))
```



Question 3: Pareto Distribution via Inverse Transformation

$f(x)$ 的 cdf 為

$$F_X(x) = \int_0^x \frac{\beta\alpha^\beta}{(t+\alpha)^{\beta+1}} dt = 1 - (\frac{\alpha}{x+\alpha})^\beta$$

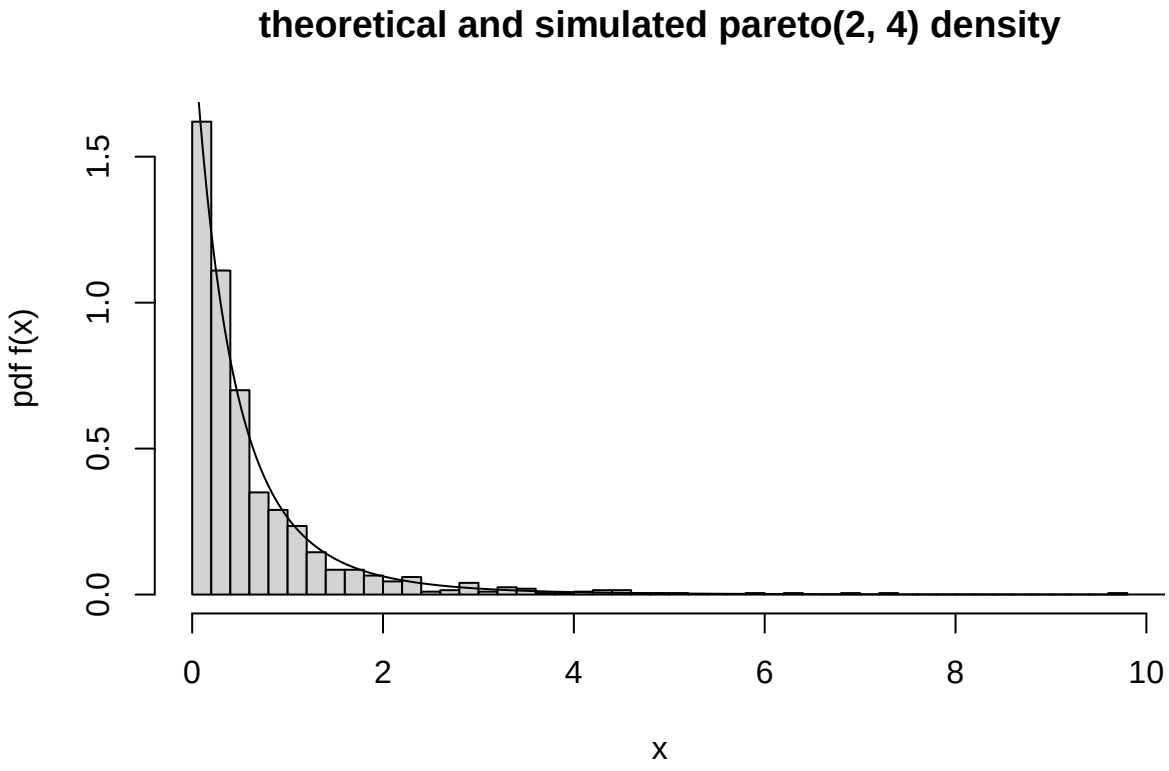
接著我們可以得到其 cdf 的 inverse 為

$$F_X^{-1}(u) = \alpha \times ((1-u)^{\frac{-1}{\beta}} - 1)$$

在順利得到 cdf 的 inverse 之後，即可透過所有 cdf 都服從 $Uniform(0,1)$ 的性質，藉由生成一堆 $Uniform(0,1)$ 的隨機樣本，再利用前面的得到 cdf inverse 對其進行轉換即可得到我們想要的結果。

```r
# cdf: 1 - (a / (x + a))^b
# cdf inverse F^-1(u): a * ((1 - u)^(-1/b) - 1)
pareto.sim <- function(a, b) {
    u <- runif(1)
    x <- a * ((1 - u)^(-1/b) - 1)
    x
}
n <- 1000
y <- rep(0, n)
for(i in 1:n) {
    y[i] <- pareto.sim(2, 4)
}

hist(y, breaks = 50, freq = F, xlab = "x", ylab = "pdf f(x)",
    main = "theoretical and simulated pareto(2, 4) density")
dpareto <- function(x, a, b) b * a^b / (x + a)^(b + 1)
z <- seq(0, 15, 0.01)
lines(z, dpareto(z, 2, 4))
```



Question 4: Wishart Distribution - Comparing Two Generation Methods

我利用了兩種方法進行比較。第一種方法為上課提及的方法，針對這個方法我使用 r 內建的 `mvrnorm` 生成多元常態分配的隨機樣本，至於 Bartlett 方法則是參照課程簡報中的演算法進行。針對這兩種方法，我利用 `system.time` 函數比較兩者運行效率。

從結果中我們可以清楚看出 Bartlett 由於減少了不必要的矩陣運算，因此在效率上明顯較一般的方法要好，執行時間約為一般法的一半。

```r
library(MASS)
rwhis.general <- function(n, Sigma) {
```

```
    x <- mvrnorm(n, mu, Sigma)
    M <- t(x) %*% x
    M
}
rwhis.bartlett <- function(d, sigma) {
    A <- matrix(rnorm(d * d), nrow = d, ncol = d)
    A[!lower.tri(A)] <- 0
    for(i in 1:d) {
        diag(A)[i] <- sqrt(rchisq(1, df = n - i + 1))
    }
    L <- t(chol(sigma))
    M <- L %*% A %*% t(A) %*% t(L)
    M
}

n <- 100 # sample size
d <- 2 # dimension
N <- 2000 # iterations
sigma <- matrix(c(1, 0.5, 0.5, 2), 2, 2, byrow = T)
mu <- numeric(d)

# general method
set.seed(100)
system.time(for(i in 1:N){
    rwhis.general(n, sigma)
})
```

```
##    user  system elapsed
##   0.071   0.005   0.322
```

```
# Bartlett decomposition method
set.seed(100)
system.time(for(i in 1:N){
    rwhis.bartlett(d, sigma)
})
```

```
##    user  system elapsed
##   0.037   0.000   0.037
```

Question 5: AFT vs. Cox Model - Monte Carlo Power Simulation

針對這題 baseline Weibull distribution 的參數設置，我採用與課程講義相同的配置，並同樣以 effect size 0.5 的方式建立 treatment 標籤。

我將 survival function 令為一個 uniform 的變數 U_i，近一步透過 inverse transform 得到其 T_i 的樣態

$$S(t|X_i) = \exp(-\lambda(t \times \exp(X_i\beta))^\nu) = U_i, Ui \sim Uniform(0,1)$$

接著推導過後可以得到

$$T_i = \exp(-X_i\beta) \times (\frac{-\ln(U_i)}{\lambda})^{\frac{1}{\nu}}$$

接著便將這個新的 T_i 代入 T_true，並執行與課程講義相同的架構即可得到結果。從結果中我們可以看到，當 censoring rate 越高，其所需要達到同樣 power 的樣本越多。除此之外，也別於 Cox PH 的每條曲線在 y 軸方向做變形，AFT 為沿著 x 軸做拉伸或壓縮。

```r
library(survival)

# Function to simulate survival data with calibrated censoring
simulate_survival_AFT <- function(n, target_censor_rate, lambda, nu, beta) {
    # 1. Covariate: Treatment Assignment (1:1 allocated)
    X <- rbinom(n, size = 1, prob = 0.5)

    # 2. Uniform draws for inverse transform
    U_t <- runif(n)

    # 3. True event times via Weibull inversion
    T_true <- ((-log(U_t) / lambda)^(1 / nu)) / exp(X * beta)

    # 4. Bisection search to calibrate tau for target censor rate
    U_c <- runif(n)
    low <- 0.1
    high <- as.numeric(quantile(T_true, 0.999))

    for(i in 1:1000) {
        mid_tau <- (low + high) / 2
        C_temp <- mid_tau * U_c
        current_rate <- mean(T_true > C_temp)

        if(abs(current_rate - target_censor_rate) < 1e-4) {
            tau <- mid_tau
            break
        } else if (current_rate > target_censor_rate) {
            low <- mid_tau
        } else {
            high <- mid_tau
        }
    }
    tau <- mid_tau

    # 5. Calculate observed variables
    C_time <- tau * U_c
    Y_obs <- pmin(T_true, C_time)
    d_event <- as.numeric(T_true <= C_time)

    data.frame(
        id  = 1:n,
        treatment = X,
        true_time = T_true,
        censor_time = C_time,
        time = Y_obs,
        status = d_event
    )
}

run_mc_power_r <- function(sample_sizes = c(50, 80, 100, 150, 200, 250, 300, 500),
                           n_sim = 1000, target_censor_rate,
                           seed = 42) {
    set.seed(seed)
```

```
    lam <- 0.04
    nu <- 1.4
    beta <- -0.7

    results <- list()

    for(N in sample_sizes) {
        p_values <- numeric(n_sim)

        for(s in 1:n_sim) {
            sim_df <- simulate_survival_AFT(n = N,
                                target_censor_rate = target_censor_rate,
                                lambda = lam,
                                nu = nu,
                                beta = beta)
            X <- sim_df$treatment
            Y_obs <- sim_df$time
            d_event <- sim_df$status

            # Cox Proportional Hazards Model fit
            fit <- coxph(Surv(Y_obs, d_event) ~ X)
            p_values[s] <- summary(fit)$coefficients["X", "Pr(>|z|)"]
        }

        power <- mean(p_values < 0.05)

        results[[as.character(N)]] <- data.frame(
            Sample_Size = N,
            Target_HR = round(exp(beta), 2),
            Censor_Rate = paste0(target_censor_rate * 100, "%"),
            Simulations = n_sim,
            Empirical_Power = round(power, 3)
        )
    }

    do.call(rbind, results)
}

# 30% censoring rate
power_table_30 <- run_mc_power_r(target_censor_rate = 0.30)
print(power_table_30)
```

```
##     Sample_Size Target_HR Censor_Rate Simulations Empirical_Power
## 50           50       0.5         30%        1000           0.738
## 80           80       0.5         30%        1000           0.934
## 100         100       0.5         30%        1000           0.970
## 150         150       0.5         30%        1000           0.999
## 200         200       0.5         30%        1000           1.000
## 250         250       0.5         30%        1000           1.000
## 300         300       0.5         30%        1000           1.000
## 500         500       0.5         30%        1000           1.000
```

```
# 50% censoring rate
power_table_50 <- run_mc_power_r(target_censor_rate = 0.50)
```

```r
print(power_table_50)
```

```
##     Sample_Size Target_HR Censor_Rate Simulations Empirical_Power
## 50           50       0.5         50%        1000           0.602
## 80           80       0.5         50%        1000           0.854
## 100         100       0.5         50%        1000           0.916
## 150         150       0.5         50%        1000           0.989
## 200         200       0.5         50%        1000           0.997
## 250         250       0.5         50%        1000           0.999
## 300         300       0.5         50%        1000           1.000
## 500         500       0.5         50%        1000           1.000
```

```r
# 70% censoring rate
power_table_70 <- run_mc_power_r(target_censor_rate = 0.70)
```

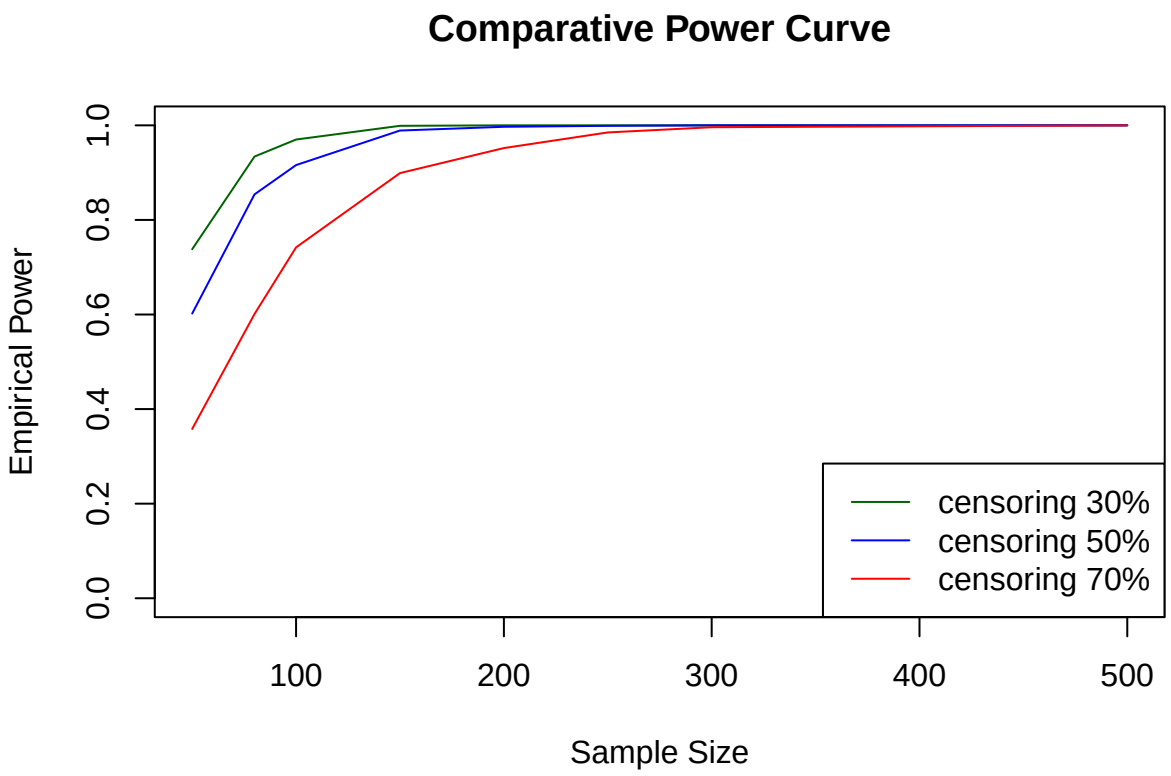
```
## Warning in coxph.fit(X, Y, istrat, offset, init, control, weights = weights, :
## Loglik converged before variable 1 ; coefficient may be infinite.
## Warning in coxph.fit(X, Y, istrat, offset, init, control, weights = weights, :
## Loglik converged before variable 1 ; coefficient may be infinite.
## Warning in coxph.fit(X, Y, istrat, offset, init, control, weights = weights, :
## Loglik converged before variable 1 ; coefficient may be infinite.
```

```r
print(power_table_70)
```

```
##     Sample_Size Target_HR Censor_Rate Simulations Empirical_Power
## 50           50       0.5         70%        1000           0.358
## 80           80       0.5         70%        1000           0.601
## 100         100       0.5         70%        1000           0.742
## 150         150       0.5         70%        1000           0.899
## 200         200       0.5         70%        1000           0.952
## 250         250       0.5         70%        1000           0.985
## 300         300       0.5         70%        1000           0.996
## 500         500       0.5         70%        1000           1.000
```

```r
plot(x = power_table_30$Sample_Size, y = power_table_30$Empirical_Power,
     type = "l", col = "darkgreen", ylim = c(0, 1), xlim = c(50, 500),
     xlab = "Sample Size", ylab = "Empirical Power",
     main = "Comparative Power Curve")
lines(x = power_table_50$Sample_Size, y = power_table_50$Empirical_Power, col = "blue")
lines(x = power_table_70$Sample_Size, y = power_table_70$Empirical_Power, col = "red")

legend("bottomright", legend = c("censoring 30%", "censoring 50%", "censoring 70%"),
       col = c("darkgreen", "blue", "red"), lty = 1, lwd = 1)
```


Comparative Power Curve
Empirical Power
0.0
0.2
0.4
0.6
0.8
1.0
100
200
300
400
500
Sample Size
censoring 30%
censoring 50%
censoring 70%